**Reliable Machine Learning with Unreliable Black Boxes** **Spring 2026**

Lecture 2: Algorithms with Predictions

*Thursday, April 30, 2026*

Instructors: A. Vijayaraghavan and V. Srinivas Scribe: Aryaman Chawla

# 1 Recap: Desiderata for Algorithms with Predictions

- **Robustness**: in the worst case scenario (when the prediction is poor), the algorithm should still perform reasonably well
- **Consistency**: in the best case scenario (when the prediction is accurate), the algorithm should take advantage of it
- **Graceful degradation**: the algorithm's performance should drop only as much as the prediction quality drops - this is sort of the middle ground between consistency and robustness

# 2 Commonly Seen Algorithmic Paradigms for Algorithms with Predictions

## 2.1 Switching

Using this strategy, the algorithm follows the prediction when the error is low, but switches to a more conservative (typically algorithmic) method when the prediction error becomes too large.

There are some strategies, such as following the prediction until a failure condition is triggered, then permanently switching to the robust (classical) algorithm or gradually shifting weight from prediction-based to classical behavior. In general, the best choice depends on the algorithm and the internal state, which might make it challenging to implement.

### Examples

- *Caching with machine learned advice* [Lykouris and Vassilvitskii, 2021], which we saw last class. Here the key is to open the hood of the marking algorithm, and show that you can switch within the algorithm, in a way that does not incur a large switching cost
- Run the prediction based algorithm and the worst case algorithm in parallel, and output the answer from the one that finishes first. You can find some extensions of this idea in [Blum and Srinivas, 2025].
- Think of prediction based algorithm and worst case algorithm as "experts" and choose which to run using *online learning with switching costs* [Cesa-Bianchi, Dekel, and Shamir, 2013]

- Some works consider a model of switching for *online algorithms* via the problem of *layered graph traversal*, which itself has a long history [Fiat, Foster, Karloff, Rabani, Ravid, and Vishwanathan, 1998]

## 2.2 Local search

In this approach, the prediction is treated as a starting point in a solution space. The algorithm then explores nearby solutions in the solution space in an informed way.

This can be very effective when the prediction is close to the optimum, because the algorithm can exploit local structure.

A challenge is that the predicted point may itself not be a *feasible* solution for the problem of interest, especially if the prediction was given without seeing the instance. Doing some sort of *feasibility projection* then becomes a key part of the algorithms with predictions task.

### Examples

- Standard recipe for *constrained convex optimization*: The input is a convex set $K \subseteq \mathcal{X}$ and convex function $f(\cdot)$, and the goal is to find

  $$\min_{x \in K} f(x).$$

  Given a prediction $p \in \mathcal{X}$ of the minimizer $x^\star$, where it may or may not be the case that $p \in K$, a standard strategy is to

  - Project $p$ onto the convex set $K$ to get $\overline{p} \in K$
  - Run gradient descent, or some other local optimization method, from $\overline{p}$ to find the minimizer in $K$ of $f(\cdot)$

  Since $\overline{p}$ is the closest point in $K$ to $p$, we have that

  $$\|p - \overline{p}\| \leq \|p - x^\star\| \quad \Longrightarrow \quad \|\overline{p} - x^\star\| \leq 2\|p - x^\star\|,$$

  so depending on the method we use, we can hope to bound the time/number of iterations to find $x^\star$ from $\overline{p}$ in terms of the prediction error $\|p - x^\star\|$. The remaining piece is to show how to do the projection onto the feasible region efficiently.

- A concrete example of the above strategy is the work of Davies, Moseley, Vassilvitskii, and Wang [2023], which solves *max flow* via a modified version of the *Ford-Fulkerson algorithm*. The key technical piece is showing how to modify the procedure to be able to start from an *infeasible* solution, and still make progress.

- Dinitz, Im, Lavastida, Moseley, and Vassilvitskii [2021] (followed up by Chen, Silwal, Vakilian, and Zhang [2022]) show how to modify a *primal-dual* algorithm for bipartite matching to incorporate predictions

- Liu and Srinivas [2024] give a strategy for *dynamic algorithms* that uses predicted information about future events. At it's core, it does some form of local search in the space of schedules of events, and shows that this can be propagated to making corrections to the algorithms output in an efficient way.

### 2.3 Bias randomness

The idea here is that instead of picking an action uniformly at random, the algorithm uses the prediction to "bias" its probability distribution. If the prediction is confident, the algorithm assigns a high probability to that choice, and if the prediction is poor, the randomness provides a safety net to match the worst-case performance.

#### Examples

- Chen, Cao, Stepin, and Chen [2025] show how to modify the *Treap* data structure to incorporate predictions. Typically the treap randomly assigns priorities to each of its elements to decide which elements will be closer to the root of the search tree. This allows it to maintain depth $O(\log n)$, with high probability, where $n$ is the number of items in the tree. [Chen, Cao, Stepin, and Chen, 2025] show a way to bias these priorities, so that items that are predicted to be accessed more often are placed closer to the root of the tree, thus minimizing lookup time.
- Niaparast, Moseley, and Singh [2025] show how to modify the *Karger-Stein* algorithm for finding global minimum cuts. The Karger-Stein algorithm proceeds by randomly contracting an edge on each iteration, until there are only two super nodes left, which are a potential mincut. They show that this succeeds at finding the mincut with some polynomially small probability, so over polynomially many trials, it can find the mincut with high probability. Niaparast, Moseley, and Singh [2025] show how to use predictions about which edges will be in the mincut to bias the probabilities of contraction, to boost the probability that the mincut will survive on any given trial.

## 3 Optimistic Prediction with Expert Advice

As another example of a algorithms with predictions, we will look at the setting of *optimistic online learning*, which was introduced by Rakhlin and Sridharan [2013]. Note that this work predates the formulation of "algorithms with predictions!" While the area of algorithms with predictions as such is relatively new, and has inspired lots of interesting and exciting work, the goal of incorporating unreliable predictions into algorithmic problems is one that has been studied under many names and paradigms for a long time.

We will follow the treatment of Steinhardt and Liang [2014], with many thanks to Aditya Bhaskara for his incredibly helpful notes on this topic.

### 3.1 Review of Classical Prediction with Expert Advice

We revisit the problem of *prediction with expert advice*, which is about making decisions using *unreliable* recommendations.

**Problem 1** (Prediction with expert advice)**.** There are $n$ "experts" that will give you advice on a sequence of $T$ decisions, which we think of as happening on $T$ "days."

On each day $t$:

- The algorithm commits to following the recommendation of an expert $i_t \in [n]$
- $\ell_t \in [0,1]^n$ is revealed, where $\ell_{t,i}$ is the loss of expert $i$s prediction on day $t$, which is a value in $[0,1]$.
- The algorithm incurs loss $\ell_{t,i_t}$

The goal of the algorithm is to choose the $i_t$ to minimize the total loss it incurs over the entire sequence of decisions:

$$\min \sum_{t \in [T]} \ell_{t,i_t}.$$

We evaluate the performance of the algorithm by measuring its *regret*.

**Definition 2** (Regret)**.** The *regret* of an algorithm for prediction with expert advice is the difference between the cost that the algorithm incurs, and the cost of the best fixed prediction in hindsight:

$$\left( \sum_{t \in [T]} \ell_{t,i_t} \right) - \left( \min_{i^\star} \sum_{t \in [T]} \ell_{t,i^\star} \right).$$

That is, we compare our algorithm to one that could see the future and make the best decisions, *but* was constrained to pick one expert to follow for the entire sequence of $T$ days.

**Example**

Suppose there are four experts $A, B, C, D$ and four days of losses. The table below records the loss of each expert on each day, along with the loss incurred by the algorithm in the example from the notes.

| Day | Algorithm action | $A$ | $B$ | $C$ | $D$ | Algorithm loss |
|---|---|---|---|---|---|---|
| 1 | choose D | 0.25 | 0.75 | 0.50 | 1.00 | 1.00 |
| 2 | choose A | 1.00 | 0.00 | 0.50 | 1.00 | 1.00 |
| 3 | choose B | 0.75 | 0.75 | 0.75 | 0.00 | 0.75 |
| 4 | choose B | 0.50 | 0.00 | 0.25 | 0.25 | 0.00 |
| Total | | 2.50 | 1.50 | 2.00 | 2.25 | 2.75 |

The best fixed expert in hindsight is expert $B$, with total loss 1.5. The regret of the algorithm is therefore 2.75 - 1.50 = 1.25.

A "good" algorithm aims for *average regret* of $O\left(\frac{1}{\sqrt{T}}\right)$. Thus as time goes on, the average regret per step goes to zero. This indicates the algorithm is effectively learning which expert to trust. This happens because we can design a classical algorithm with a loss

$$\leq (\text{best expert's total loss}) + O(\sqrt{T \log(n)}.$$

**Theorem 3.** *We can design algorithms for the prediction with expert advice problem that achieve regret* $O\left(\sqrt{T \log n}\right)$.

This problem, and algorithms for it, have been studied in many forms and rediscovered in many fields from as early as the 1950s. We will assume some background about this problem and algorithms for it. In particular, the algorithm for optimistic online learning that we will see is based on the *Hedge* algorithm and the *Multiplicative Weight Update* algorithm. A good resource to learn more about these is the textbook of Arora, Hazan, and Kale [2012].

### 3.2 Optimistic Setting

In the classical setting, every "expert" is a potentially completely incorrect, and we must hedge against the possibility that following them could lead us astray. However, in some settings we may already have a better idea of which decisions to follow.

One way to model this is as follows:

**Problem 4** (Optimistic Prediction with Expert Advice)**.** Same as *prediction with expert advice*, except that on each day $t$, before making it's decision, the algorithm gets to see a vector $m_t \in [0,1]^n$, such that $m_{t,i}$ is a *prediction* of the loss of expert $i$ on day $t$.

This allows us to design an algorithm with the following guarantee.

**Theorem 5** (Guarantee for Optimistic Prediction with Expert Advice)**.** *For an input parameter $\eta \in (0,1]$, Algorithm 1 chooses experts $i_t$ to achieve expected total loss*

$$\mathbb{E}\left[\sum_{t\in[t]} \ell_{t,i_t}\right] \leq \left(\sum_{t\in[T]} \ell_{t,i}\right) + \frac{\log n}{\eta} + \eta \sum_{t\in[T]} \varepsilon_{t,i}^2,$$

*for all $i \in [T]$ simultaneously, where $\varepsilon_{t,i}$ is the* prediction error *of $m_t$ on expert $i$:*

$$\varepsilon_t := \ell_t - m_t.$$

**Algorithm 1** Optimistic Prediction with Expert Advice

**Require:** Parameter $\eta \in (0,1]$

$w_{0,i} := 1$, for all experts $i \in [n]$
**for** day $t$ **do**
  Predict $p_{t,i} \propto w_{t,i} \cdot e^{-\eta\ m_{t,i}}$
    // play a randomized strategy over experts, with probabilities as above
  Receive loss vector $\ell_t$, calculate prediction error $\varepsilon_t := \ell_t - m_t$
  Set $w_{(t+1),i} := w_{t,i}\ e^{-\eta\ m_{t,i}}\ (1 - \eta\varepsilon_{t,i})$

#### Interpretation

- The prediction with expert advice problem already captures the setting of following unreliable information. One thing we could have done is incorporate the prediction by simulating an $(n+1)$th expert, who always follows the lowest cost expert according to $m_t$. Using our classical guarantee, when the predictons are perfect, this would give us $O\left(\sqrt{T \log n}\right)$ regret with respect to this simulated expert. The above result gives lower regret of only $\log n$ with respect to this expert, if the predictions are correct.

- Rakhlin and Sridharan [2013] also consider other weaker models of advice, such as predicting only the best expert on each day
- In the language of algorithms with predictions, the above guarantee ensures *consistency* and *graceful degradation* with respect to the predictions. To also achieve a guarantee of *robustness*, we can easily switch to running a worst case algorithm for prediction with expert advice, if we ever detect that the error of the optimistic algorithm is too high.

We will see the analysis of this algorithm in the next class.

## References


Sanjeev Arora, Elad Hazan, and Satyen Kale. The multiplicative weights update method: a meta-algorithm and applications. *Theory of Computing*, 8(6):121–164, 2012. doi: 10.4086/toc.2012.v008a006. URL https://theoryofcomputing.org/articles/v008a006.

Avrim Blum and Vaidehi Srinivas. Competitive strategies to use "warm start" algorithms with predictions. In *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 3775–3801, 2025. doi: 10.1137/1.9781611978322.127. URL https://arxiv.org/abs/2405.03661.

Nicolò Cesa-Bianchi, Ofer Dekel, and Ohad Shamir. Online learning with switching costs and other adaptive adversaries. In *Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 1*, NIPS'13, page 1160–1168, Red Hook, NY, USA, 2013. Curran Associates Inc. URL https://arxiv.org/abs/1302.4387.

Jingbang Chen, Xinyuan Cao, Alicia Stepin, and Li Chen. On the power of learning-augmented search trees. In *Proceedings of the 42nd International Conference on Machine Learning*, ICML'25. JMLR.org, 2025. URL https://arxiv.org/abs/2211.09251.

Justin Chen, Sandeep Silwal, Ali Vakilian, and Fred Zhang. Faster fundamental graph algorithms via learned predictions. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 3583–3602. PMLR, 17–23 Jul 2022. URL https://arxiv.org/abs/2204.12055.

Sami Davies, Benjamin Moseley, Sergei Vassilvitskii, and Yuyan Wang. Predictive flows for faster ford-fulkerson. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org, 2023. URL https://arxiv.org/abs/2303.00837.

Michael Dinitz, Sungjin Im, Thomas Lavastida, Benjamin Moseley, and Sergei Vassilvitskii. Faster matchings via learned duals. In A. Beygelzimer, Y. Dauphin, P. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, 2021. URL https://arxiv.org/abs/2107.09770.

Amos Fiat, Dean P. Foster, Howard Karloff, Yuval Rabani, Yiftach Ravid, and Sundar Vishwanathan. Competitive algorithms for layered graph traversal. *SIAM Journal on Computing*, 28(2):447–462, 1998. doi: 10.1137/S0097539795279943. URL https://deanfoster.net/research/layered_graph_traversal.pdf.

Quanquan C. Liu and Vaidehi Srinivas. The predicted-updates dynamic model: Offline, incremental, and decremental to fully dynamic transformations. In Shipra Agrawal and Aaron Roth, editors, *Proceedings of Thirty Seventh Conference on Learning Theory*, volume 247 of *Proceedings of Machine Learning Research*, pages 3582–3641. PMLR, 30 Jun–03 Jul 2024. URL https://arxiv.org/abs/2307.08890.

Thodoris Lykouris and Sergei Vassilvitskii. Competitive caching with machine learned advice. *J. ACM*, 68(4), July 2021. ISSN 0004-5411. doi: 10.1145/3447579. URL https://arxiv.org/abs/1802.05399.

Helia Niaparast, Benjamin Moseley, and Karan Singh. Faster global minimum cut with predictions. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 46305–46319. PMLR, 13–19 Jul 2025. URL https://arxiv.org/abs/2503.05004.

Alexander Rakhlin and Karthik Sridharan. Online learning with predictable sequences. In Shai Shalev-Shwartz and Ingo Steinwart, editors, *Proceedings of the 26th Annual Conference on Learning Theory*, volume 30 of *Proceedings of Machine Learning Research*, pages 993–1019, Princeton, NJ, USA, 12–14 Jun 2013. PMLR. URL https://arxiv.org/abs/1208.3728.

Jacob Steinhardt and Percy Liang. Adaptivity and optimism: An improved exponentiated gradient algorithm. In Eric P. Xing and Tony Jebara, editors, *Proceedings of the 31st International Conference on Machine Learning*, volume 32 of *Proceedings of Machine Learning Research*, pages 1593–1601, Bejing, China, 22–24 Jun 2014. PMLR. URL https://proceedings.mlr.press/v32/steinhardtb14.html.